

ROADRESQ

Incident Response Plan

Security • Payments • Infrastructure • Provider Abuse • Operational Incidents

Document Type: Production Business / Security / Operations Runbook

Version: 1.0 **Status:** Production Readiness Baseline

Date: 04 September 2026

Purpose: Define how RoadResQ detects, classifies, contains, communicates, recovers from, and learns from security, payment, database, availability, provider-abuse, safety, and other production incidents.

Important: This is an operational/product requirement document, not legal advice. Regulatory notification, law-enforcement interaction, contractual obligations, and other legal decisions must be handled with qualified counsel where required.

1. Document Control

Field	Requirement
Document	RoadResQ Incident Response Plan
Version	1.0
Owner	RoadResQ Engineering + Security + Operations
Applies To	Customer app/web, provider systems, admin, APIs, database, Redis, workers, realtime, payments, storage, integrations and support operations
Primary Goal	Restore safe service quickly while preserving evidence, protecting users and preventing recurrence
Incident Commander	Designated senior responder for major incidents
Review Cadence	At least quarterly and after every material incident

2. Incident Response Principles

- **Safety first:** Protect customers, providers, responders, and vehicles before optimizing availability or revenue.
- **Contain before cleanup:** Stop ongoing harm and preserve evidence before making destructive changes.
- **One incident commander:** Major incidents require a single accountable coordinator even when many teams participate.
- **Least privilege:** Incident access is granted only to the people and systems needed for response.
- **Communicate facts:** Separate confirmed facts, hypotheses, and unknowns.
- **Prefer reversible actions:** Use feature flags, traffic controls, isolation, rollback, and scoped blocks where possible.
- **Every incident teaches:** Complete a post-incident review and track corrective actions to closure.

3. Incident Severity Model

Severity	Definition	Examples	Initial Target
SEV-0 Critical	Immediate threat to life/safety, catastrophic security compromise, uncontrolled financial impact, or complete platform failure	Major safety incident; active account/data compromise; catastrophic payment or DB corruption	Immediate response; executive escalation
SEV-1 High	Major customer/provider impact, significant security or financial risk, or core service unavailable	Emergency matching down; database outage; widespread payment failure; major provider abuse	Page on-call immediately
SEV-2 Medium	Material degradation with workaround or limited scope	Regional API degradation; notification failure; isolated payment reconciliation issue	Respond during active support window / on-call
SEV-3 Low	Limited impact, no significant safety/security/financial risk	Minor UI issue; isolated non-critical integration error	Normal engineering/support workflow

4. Incident Categories

Category	Examples	Primary Response Focus
Security	Account takeover, credential compromise, data exposure, malware, unauthorized admin action	Containment, access revocation, evidence, scope assessment
Payment	Gateway outage, webhook failure, duplicate charge, payment pending, reconciliation mismatch	Protect financial state, idempotency, reconciliation, customer communication

Category	Examples	Primary Response Focus
Database	PostgreSQL outage, corruption, migration failure, replication issue, data inconsistency	Protect data, failover/recovery, PITR, integrity validation
Infrastructure	API outage, Redis failure, worker/queue failure, storage/CDN outage	Restore availability, isolate dependency, rollback/failover
Realtime / Location	WebSocket outage, stale provider GPS, incorrect location visibility	Safety, privacy, stale-state handling, recovery
Provider Abuse	Fraud, harassment, unsafe conduct, fake completion, price manipulation, document fraud	Suspend risk, protect customers, investigate, preserve evidence
Customer Abuse	Fake emergency requests, harassment, fraud, payment abuse, system misuse	Protect providers/platform, investigate, restrict account
Safety	Accident, unsafe service site, threat, injury, emergency misuse	Immediate safety escalation and emergency-service direction
Privacy	Unauthorized access, excessive exposure, deletion failure, sensitive-data leak	Contain exposure, access review, privacy assessment
Third-Party	Maps, OTP/SMS, payment, cloud, analytics dependency outage	Fallback, degradation mode, vendor escalation

5. Roles and Responsibilities

Role	Core Responsibilities
Incident Commander (IC)	Owns incident coordination, severity, priorities, decisions, timeline, escalation and closure.
Technical Lead	Leads diagnosis, containment, rollback, recovery and technical verification.
Security Lead	Handles security/privacy incidents, evidence preservation, access revocation and threat assessment.
Payments/Finance Lead	Owns payment-state protection, reconciliation, refunds, payouts and financial impact.
Database/Infrastructure Lead	Owns DB, cache, queues, storage, deployment, backup and infrastructure recovery.
Operations Lead	Coordinates provider/customer operations, support queues, safety and service continuity.
Communications Lead	Maintains internal updates and approved customer/provider status communications.
Support Lead	Provides user-facing case handling, evidence collection and escalation.
Legal/Privacy Contact	Advises on legal, regulatory, privacy, contractual and notification obligations.
Scribe	Maintains incident timeline, decisions, actions, owners and evidence references.

6. Standard Incident Lifecycle

- **1. Detect:** Alert, support report, user report, provider report, monitoring signal, vendor notice, audit finding, or staff discovery.
- **2. Triage:** Validate the issue, identify affected systems/users and assign preliminary severity.
- **3. Declare:** Create the incident record, assign IC and responders, establish communication channel.
- **4. Contain:** Stop spread or ongoing impact using access revocation, feature disablement, traffic controls, provider suspension, payment holds, isolation or rollback.
- **5. Investigate:** Establish timeline, scope, root cause, affected records and dependency behavior.
- **6. Recover:** Restore services/data and verify integrity before returning to normal operation.

- **7. Communicate:** Send accurate, appropriate updates to customers, providers, support, management and external parties.
- **8. Close:** Confirm monitoring stability and business recovery; document final status.
- **9. Learn:** Conduct post-incident review and track preventive actions.

7. Detection and Intake Requirements

- Monitor API availability, latency, error rate, database health, Redis, queues, WebSockets, payments, provider matching, location freshness, authentication and security signals.
- Support must have a clear escalation path for reports involving safety, fraud, payment loss, privacy, account takeover or platform-wide failure.
- Every incident must have a unique incident ID.
- Record detection timestamp, reporter, affected feature, preliminary severity, symptoms and current impact.
- Avoid putting unnecessary personal or payment data into incident channels or free-text logs.
- Preserve relevant logs, audit events, traces, payment events, booking state history and infrastructure evidence.

8. Security Incident Response

8.1 Account Compromise / Credential Theft

- Assess whether the account, session, OTP, API token or admin credential remains compromised.
- Revoke active sessions/tokens and reset affected credentials where appropriate.
- Temporarily restrict high-risk actions such as payout changes, provider-document changes or payment actions.
- Review audit logs for login, role, profile, vehicle, booking, payment and payout changes.
- Identify affected users and determine whether unauthorized data access occurred.
- Escalate privacy/legal assessment when personal data may have been exposed.

8.2 Data Exposure / Unauthorized Access

- Stop the exposure path: revoke access, disable endpoint, rotate credentials, change permissions, isolate storage or block traffic.
- Preserve evidence before deleting or modifying compromised resources where feasible.
- Determine data categories, users, time window, access method and likely recipients.
- Review object-level authorization and privileged access logs.
- Assess whether backups, caches, exports or third-party systems were also affected.
- Use legal/privacy guidance for required notifications and regulatory actions.

8.3 Malicious Code / Infrastructure Compromise

- Isolate affected workloads without unnecessarily destroying forensic evidence.
- Rotate potentially exposed secrets and credentials.
- Check deployment history, image provenance, CI/CD logs and administrative access.
- Rebuild from a known-good source/image rather than trusting a potentially compromised host.
- Verify database and object-storage integrity before restoration.
- Increase monitoring after recovery and document the initial compromise vector.

9. Payment Incident Response

9.1 Payment Gateway Outage

- Confirm whether the failure is RoadResQ, gateway, bank/network, or client-side.
- Stop repeated retry loops that could create duplicate charges.
- Keep booking and payment state transitions idempotent.
- Mark payments as pending/unknown when confirmation cannot be trusted.
- Do not manually mark payments successful without verifiable provider/gateway evidence.
- Communicate degraded payment behavior and provide safe retry guidance.

9.2 Duplicate Charge / Duplicate Payment

- Freeze automated duplicate retry behavior.
- Correlate internal payment intent, gateway transaction ID, webhook event ID and invoice.
- Determine whether the customer was charged once or multiple times.
- Initiate the approved refund/adjustment workflow when a duplicate charge is confirmed.
- Record the correction in the internal ledger and reconciliation process.
- Prevent recurrence through idempotency keys, unique constraints and webhook deduplication.

9.3 Payment Webhook Failure

- Treat webhook events as potentially duplicated, delayed or out of order.
- Use immutable event IDs and idempotent processing.
- Reconcile uncertain transactions against the payment provider.
- Do not rely solely on client-side success messages for final financial state.
- Replay safely only after confirming event-processing behavior.

10. Database Incident Response

10.1 Database Outage

- Identify whether the issue is connection saturation, instance failure, network failure, lock contention, storage, migration, or provider outage.
- Protect data from unsafe writes when integrity is uncertain.
- Fail over according to the production architecture when an approved replica is available.
- Monitor active bookings and payment states for inconsistent transitions.
- Restore application traffic gradually after database health and consistency are verified.

10.2 Data Corruption / Bad Migration

- Stop or roll back the change if safely reversible.
- Identify exact migration/version and affected tables/records/time window.
- Preserve current state for forensic comparison.
- Use backup/PITR recovery where required.
- Validate referential integrity, booking state, invoices, payments, payouts and critical audit records.
- Document any data reconstruction or manual correction.

11. Provider Abuse Response

Scenario	Immediate Action	Investigation / Outcome
Threats / violence / unsafe conduct	Protect customer; suspend provider from new jobs; escalate safety issue	Review evidence, support reports, communications and job history; determine suspension/offboarding
Fraud / fake jobs	Hold risky activity and payouts where authorized	Compare GPS, booking, payment, completion and account patterns
Price manipulation	Pause disputed settlement where appropriate	Review estimate, approval, invoice and communication records
Document fraud	Suspend verification/activation	Re-verify evidence; reject/suspend/offboard as appropriate
Repeated poor service	Restrict or review provider	Analyze complaints, ratings, cancellations and completion quality
Location manipulation	Restrict affected account/job actions	Compare GPS freshness, timestamps and operational records

12. Customer Abuse Response

- Fake emergency requests or repeated fraudulent dispatches may trigger request restrictions and account review.
- Harassment, threats, theft, or unsafe behavior toward providers must be escalated through support/operations.
- Payment fraud and chargeback abuse should be handled with finance and risk controls.
- Do not expose internal fraud-detection rules to users in ways that facilitate circumvention.
- Preserve evidence and document decisions with proportionality and appeal/review paths where applicable.

13. Safety Incident Response

- If immediate danger, injury, fire, crime, or another public emergency exists, direct the person to the appropriate emergency/public service and prioritize physical safety.
- Do not delay emergency assistance while troubleshooting RoadResQ systems.
- Record only necessary safety information and restrict access to sensitive case details.
- Freeze relevant booking/provider state when necessary to prevent further unsafe dispatch.
- Collect incident evidence through approved support/claims channels.
- Escalate serious incidents to operations leadership and legal/insurance contacts as appropriate.

14. Realtime / Location Incident Response

- If provider GPS becomes stale, show stale/unknown status rather than presenting outdated coordinates as live.
- If WebSocket connections fail, fall back to safe polling or last-known-state behavior where supported.
- If location visibility is accidentally broader than intended, disable the affected channel and revoke access.
- Do not store continuous location history unless the product and privacy requirements justify it.
- After recovery, verify that only authorized customer/provider/admin roles can view location.

15. Service / Infrastructure Outage Playbook

Component	First Checks	Recovery Options
API	Error rate, latency, deployment, DB connections	Rollback, scale, disable feature, restore dependency
PostgreSQL/PostGIS	Connections, CPU/storage, locks, replication	Failover, connection control, recovery/PITR

Component	First Checks	Recovery Options
Redis	Memory, connectivity, eviction, persistence	Failover/restart, degrade cache-dependent features
Workers/Queues	Backlog, failures, poison jobs	Pause/retry safely, inspect idempotency, drain/replay
WebSockets	Connection count, broker, auth	Restart/scale, fallback polling
Object Storage	Access errors, permissions, vendor status	Vendor escalation, restore access
Maps	API errors, quota, latency	Fallback provider/manual location where supported
Payments	Gateway status, webhooks, transaction state	Degraded mode, reconciliation, vendor escalation
Notifications	Push/SMS/email failures	Retry safely, fallback channel, queue replay

16. Communication Plan

16.1 Internal Updates

- Major incidents should have a dedicated incident channel/record.
- Update cadence should reflect severity; SEV-0/SEV-1 incidents require frequent concise updates.
- Every update should include current impact, confirmed cause if known, actions underway, risks, next decision point, and owner.
- Never publish secrets, authentication tokens, full payment credentials, unnecessary personal data, or sensitive forensic details.

16.2 Customer / Provider Communications

- State what is affected, what users should do, and whether action is required.
- Do not speculate about root cause before confirmation.
- For payment incidents, distinguish payment submitted, pending, failed, and confirmed states.
- For location/realtime issues, clearly communicate that tracking may be delayed or unavailable.
- Safety incidents receive operational priority over routine service messaging.

17. Evidence and Incident Record

Required Field	Examples
Incident ID	INC-2026-0001
Detected At	Timestamp + timezone
Severity	SEV-0 / SEV-1 / SEV-2 / SEV-3
Affected Systems	API, DB, payments, provider dispatch, etc.
Affected Users	Approximate count and affected segment where known
Timeline	Detection, declaration, containment, recovery, closure
Evidence	Logs, traces, audit records, webhook IDs, screenshots, vendor case IDs
Actions	Action, owner, start/end time, result
Customer Impact	Bookings, payments, location, safety, access
Financial Impact	Charges, refunds, payouts, revenue exposure
Root Cause	Confirmed technical/operational cause

Required Field	Examples
Contributing Factors	Process, monitoring, configuration, dependency, human factors
Corrective Actions	Owner, priority, due date, verification

18. Recovery Verification

- Health checks are green and stable for the required observation period.
- Core booking journey works end-to-end: request → match → accept → tracking → arrive → inspect → approve → repair → pay → review.
- Payment states reconcile correctly with gateway/provider records.
- No unexpected duplicate jobs, charges, payouts, notifications or state transitions exist.
- Database integrity and critical records are validated.
- Provider/customer access and authorization are functioning correctly.
- Security controls and audit logging are restored.
- Queues/backlogs are stable and no unsafe retries remain.
- Support receives a clear resolution message and handling instructions.

19. Post-Incident Review

- Conduct a review for SEV-0/SEV-1 incidents and material SEV-2 incidents.
- Focus on systems and processes rather than blame.
- Document timeline, root cause, contributing factors, what worked, what failed, and what should change.
- Create corrective/preventive actions with named owners and deadlines.
- Identify missing monitoring, tests, runbook steps, security controls, UX states or operational procedures.
- Track actions until verified complete; do not close merely because a ticket exists.

20. Incident Metrics

Metric	Purpose
MTTD — Mean Time to Detect	Measures monitoring/reporting effectiveness
MTTA — Mean Time to Acknowledge	Measures response readiness
MTTC — Mean Time to Contain	Measures ability to stop impact
MTTR — Mean Time to Recover	Measures restoration effectiveness
Repeat Incident Rate	Measures whether corrective actions work
Customer Impact Duration	Measures actual user disruption
Financial Exposure	Measures payment/revenue risk
Security Incident Scope	Measures affected accounts/data/systems
Action Closure Rate	Measures post-incident improvement discipline

21. Tabletop Exercises & Drills

- Run quarterly tabletop exercises covering at least one security incident, one payment incident, one database outage, and one provider-abuse/safety scenario.
- Test contact/escalation paths, incident declaration, evidence handling, customer communications and recovery decisions.
- Perform controlled restore/failover exercises according to the Backup & Disaster Recovery Plan.
- Record drill findings and convert material gaps into tracked engineering/operations actions.
- Do not use production customer data unnecessarily during exercises.

22. Production Incident Readiness Checklist

- On-call ownership and escalation contacts are defined.
- Incident management channel/tool is available.
- Monitoring and alerting cover critical customer/provider journeys.
- Logs, metrics, traces and audit records have usable correlation IDs.
- Backups and recovery procedures are tested.
- Payment reconciliation procedures are documented.
- Provider suspension and account restriction controls are available.
- Feature flags/kill switches exist for high-risk functionality where appropriate.
- Emergency and safety escalation contacts are defined.
- Vendor support/escalation paths are documented.
- Secrets can be rotated quickly.
- Admin actions are auditable.
- Support has incident communication templates and escalation rules.
- Post-incident action tracking exists.

23. Quick Reference — First 15 Minutes

Time	Action
0–2 min	Acknowledge alert/report; assess immediate safety/security/financial impact.
2–5 min	Assign severity and Incident Commander; open incident record/channel.
5–10 min	Identify affected systems/users; start containment; preserve evidence.
10–15 min	Assign technical/security/operations leads; publish first internal status; decide rollback, isolation, failover, suspension or degraded mode.

24. Definition of Done for an Incident

- Customer/provider impact has ended or an approved stable degraded state is active.
- Security and safety risks are contained.
- Payment and booking states are reconciled.
- Critical data integrity is verified.
- Monitoring confirms stability.
- Support and relevant stakeholders have been informed.

- Incident timeline and evidence are complete.
- Root cause or current best-known cause is documented.
- Corrective actions have owners and due dates.
- Required legal/privacy/contractual assessment has been escalated.
- Post-incident review is scheduled or completed according to severity.

25. Relationship to Other RoadResQ Documents

This Incident Response Plan should operate with the RoadResQ Security Architecture & Threat Model, Production Operations Runbook, Monitoring & Observability Plan, Backup & Disaster Recovery Plan, Payment & Settlement Specification, Customer Support & Dispute SOP, Provider Operations & Verification SOP, Privacy & Data Governance Document, API Specification, System Architecture, Testing & QA Plan, Deployment/DevOps Plan, and UX State & Edge-Case Document.

When documents conflict, the active production architecture, approved security controls, payment integrity requirements, safety procedures, and legally reviewed policies should be reconciled through formal change management before implementation.

26. Final Standard

RoadResQ is production-ready for incident response only when incidents can be detected, escalated, contained, investigated, recovered, communicated, and learned from without relying on undocumented individual knowledge. Critical controls must be executable by an on-call team under pressure, with clear ownership, evidence preservation, safe fallbacks, tested recovery paths, and auditable decisions.